

## **DnA Realty: A Real-Estate Office Database**

Anuraag Dasari and Dhanush Annoji; Database Systems Project

### **Overview:**

This project implements a small relational database for the operations of a real estate office. The office needs to keep track of its agents, the buyers and sellers it works with, the homes currently on the market, and the homes that have recently sold. The goal was to move through the full lifecycle of a database project, starting from conceptual design, moving to a logical relational schema, and ending with a working implementation that can answer real questions about the office's business. The application focuses on residential homes rather than commercial real estate, and it is built so that a user who is not technical can search listings, record sales, and review agent performance through a simple interface.

### **Conceptual Design**

We began with an Entity-Relationship model. The design has five entity sets. Agents, sellers, and buyers each represent a category of person the office deals with, and each has its own attributes such as name and phone number, with agents also carrying an email address. Properties represent the homes themselves and hold the bulk of the descriptive data, including address, city, school district, number of bedrooms and bathrooms, whether the home has a pool, the list price, the listing date, and an optional photo. Sales represent completed transactions and store the sale price and sale date.

Every entity has its own primary key, so none of them are weak entities. The relationships connect these entities in the ways the business requires. A seller owns properties, an agent lists properties, an agent sells in a sale, and a buyer buys in a sale. Each of these is a one to many relationship, since for example one agent can list many homes but each home has a single listing agent. The relationship between properties and sales is one to one, because a home can be sold at most once. One important design detail is that an agent plays two different roles in a sale. An agent can be the selling agent, and separately an agent can be the buyer's agent. The buyer's agent is optional, since a buyer does not always bring their own representation.

### **Logical Design**

We converted the E-R model into a relational schema implemented in SQLite. Each entity became a table, and the relationships were captured through foreign keys. The properties table references both a seller and a listing agent. The sales table references a property, a buyer, and two agents, one for the selling agent and one for the optional buyer's agent, which is how the two roles from the conceptual model appear in the tables.

The most significant design decision is how the schema distinguishes a home that is for sale from one that has sold. Rather than adding a status column that a user could forget to update, we placed a unique constraint on the property reference in the sales table. Because a property can appear in the sales table at most once, and because a home that has never sold simply has no matching sales row, the for sale status is derived directly from the data. This keeps the database

honest, since there is no separate flag that can fall out of sync with reality. We also added several constraints and indices, including a check that the pool value is either zero or one, not null requirements on the essential columns, and indices on the columns used most often in the required queries.

### **Implementation**

The application is organized so that responsibilities are cleanly separated. The schema file defines the tables and constraints, and a seeding script populates the database with synthetic sample data. We generated our own data rather than using real listings, and the data was chosen so that each required query returns a meaningful, non-empty answer. Only a few houses carry a photo, including the most expensive one, and the rest have no image, which matches the project guidance. A data layer module holds all of the SQL, so that the interface never builds queries directly. The interface itself is a graphical application with tabs for searching properties, reviewing agent performance, and managing the office by adding agents or recording sales.

### **Queries**

The system answers all of the required questions. It can find homes for sale in a given city within a price range, and homes for sale in a given school district that have at least four bedrooms and no pool. It can identify the agent who sold the most property by total dollar value in a given year, and it can report, for each agent, the average sale price and the average number of days a property spent on the market. It can display the most expensive home in the database along with its photo. Finally, it can record a new sale against a listed home and add a new agent to the office. The record sale operation also protects against selling the same home twice by refusing to insert a second sales row for a property that is already sold.

### **Reflection**

Building this project gave us practical experience with the full arc of database design. The part that proved most valuable was learning to let the schema enforce business rules rather than relying on the application to remember them. The unique constraint that derives for sale status is a small choice, but it prevents a whole class of data errors. If we extended the project further, we could add relationship attributes such as a commission rate on the listing, or track price history over time. Overall the project produced a complete and correct database that supports every operation the office asked for.

## Table sizes

| Relation   | count(*) |
|------------|----------|
| agents     | 5        |
| sellers    | 6        |
| buyers     | 5        |
| properties | 11       |
| sales      | 5        |

## Queries

**Query (A). Find addresses of homes for sale in the city “Bethlehem” costing between \$200,000 and \$250,000.**

```
SELECT p.address, p.city, p.list_price
FROM   properties p
LEFT JOIN sales s ON s.property_id = p.property_id
WHERE  s.sale_id IS NULL      -- still for sale
      AND p.city = 'Bethlehem'
      AND p.list_price BETWEEN 200000 AND 250000;
```

**Answer:** 12 Maple St (\$215,000) and 48 Cedar Ave (\$239,000). A \$172,000 Bethlehem home is correctly excluded by the price band.

**Query (B). Find addresses of homes for sale in the school district “Parkland” with 4+ bedrooms and no swimming pool.**

```
SELECT p.address, p.bedrooms, p.has_pool, p.list_price
FROM   properties p
LEFT JOIN sales s ON s.property_id = p.property_id
WHERE  s.sale_id IS NULL
      AND p.school_district = 'Parkland'
      AND p.bedrooms >= 4
      AND p.has_pool = 0;
```

**Answer:** 5 Willow Way (5 bed) and 77 Orchard Ln (4 bed). A Parkland home with a pool and a 3-bedroom one are both excluded.

**Query (C). Find the agent who has sold the most property in the year 2004 by total dollar value.**

```
SELECT a.name, SUM(s.sale_price) AS total_value, COUNT(*) AS num_sales
FROM   sales s
JOIN   agents a ON a.agent_id = s.selling_agent_id
WHERE  strftime('%Y', s.sale_date) = '2004'
GROUP BY a.agent_id
ORDER BY total_value DESC
LIMIT 1;
```

**Answer:** Bob Saget: \$533,000 in total across 2 sales in 2004.

**Query (D). For each agent, the average selling price of properties sold in 2004 and the average time each property was on the market.**

```
SELECT a.name,
       ROUND(AVG(s.sale_price), 2) AS avg_price,
```

```

        ROUND(AVG(julianday(s.sale_date)
                  - julianday(p.listing_date)), 1) AS avg_days_on_market
FROM    sales s
JOIN    properties p ON p.property_id = s.property_id
JOIN    agents a      ON a.agent_id = s.selling_agent_id
WHERE   strftime('%Y', s.sale_date) = '2004'
GROUP BY a.agent_id;

```

**Answer:** Bob Saget: avg \$266,500, 75 days. Kevin Nguyen: avg \$201,000, 56 days. Phil Dunphy: avg \$168,000, 47 days. Time on market is sale\_date minus listing\_date, so the design stores both dates.

**Query (E). Show a picture of the most expensive house(s) in the database.**

```

SELECT address, city, list_price, photo
FROM    properties
WHERE   list_price = (SELECT MAX(list_price) FROM properties);

```

**Answer:** 101 Summit Dr, Bethlehem: \$640,000. Its photo is stored as a BLOB; the GUI reads the bytes back and renders the image in the property detail window.

**Query (F). Record the sale of a property that had been listed as available (price, buyer, selling agent, buyer's agent if any, and date).**

```

-- the program first checks the property is not already sold
INSERT INTO sales (property_id, buyer_id, selling_agent_id,
                  buyer_agent_id, sale_price, sale_date)
VALUES (7, 1, 2, NULL, 380000, '2005-09-01');

```

**Answer:** 22 Pine Rd is sold to Harry Potter, selling agent Bob Saget, no buyer's agent, for \$380,000 on 2005-09-01. A new sales row is inserted and the property immediately drops out of the “for sale” results and reports.

**Query (G). Add a new agent to the database.**

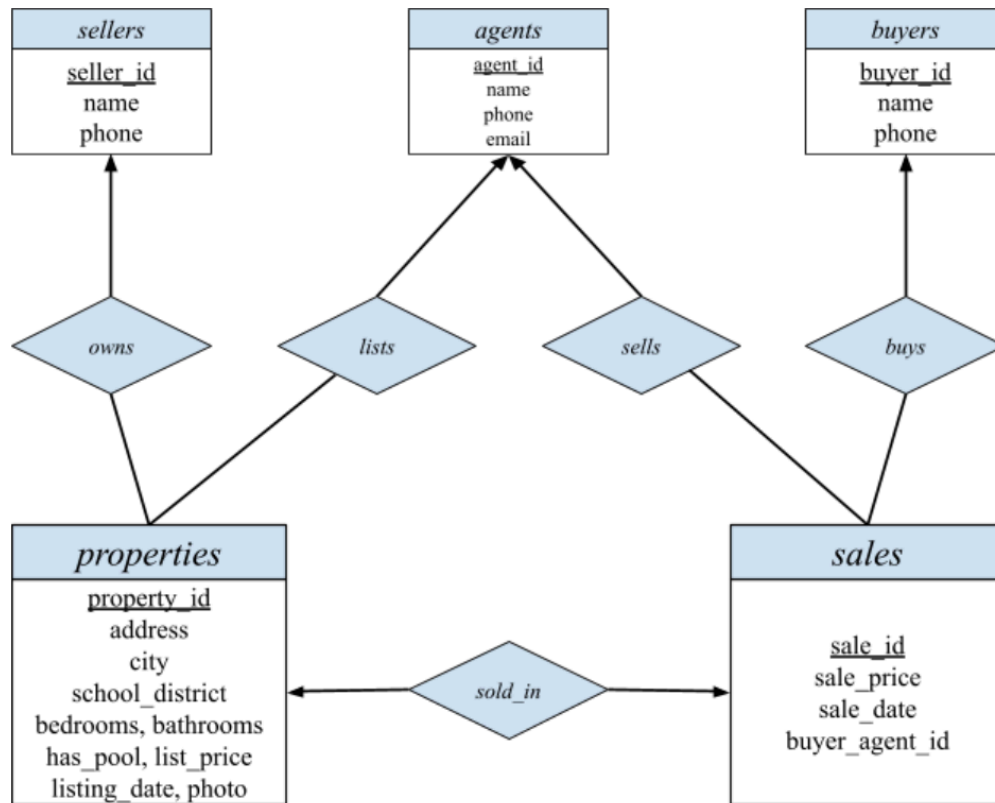
```

INSERT INTO agents (name, phone, email)
VALUES ('Bart Simpson', '404-215-8936', 'bartyboy@realty.com');

```

**Answer:** A new agents row is created (email must be unique), and the agent appears in the Current agents list.

**E-R Diagram:**



## Relational Schema:

